LIGATE

PDP 2026
34th Euromicro International
Conference on Parallel, Distributed,
and Network-Based Processing

Optimizing the LiGen Drug Discovery Pipeline For Intel Max GPUs

Saleh JAMALI GOLZAR[1], *Lorenzo CARPENTIERI*[1], *Antonio DE CARO*[1], *Biagio COSENZA*[1], *Davide GADIOLI*[2], *Gianmarco ACCORDI*[2], *Gianluca PALERMO*[2], *Federico FICARELLI*[3], *Daniele GREGORI*[4], *Andrea R. BECCARI*[5]

1 University of Salerno, Italy
2 Polytechnic University of Milan, Italy
3 CINECA, Italy
4 E4 Computer Engineering, Italy
5 Dompé Farmaceutici SpA, Italy

EuroHPC
Joint Undertaking

This project has received funding from the European High-Performance Computing Joint Undertaking (JU) under grant agreement No 956137. The JU receives support from the European Union's Horizon 2020 research and innovation programme and from Italy, Sweden, Austria, Czech Republic, and Switzerland.

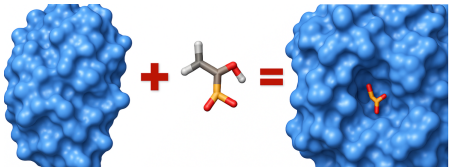
Outline

1. **Motivation & Background**
   Virtual screening, LiGen pipeline, vendor lock-in, GPU hardware diversity
2. **Proposed Approach**
   Performance-portable heuristic (SYCL-heur) + Intel-specific optimizations (SYCL-intel)
3. **Experimental Evaluation**
   Throughput results on Intel Max 1100 & cross-platform roofline analysis
4. **Conclusion**

Motivation & Background

Virtual Screening

- **Ligand**: small molecule (<100 atoms) – drug candidate
- **Protein**: drug target with binding pockets
- **Docking**: fit ligand into protein pocket
  - Protein = rigid body, ligand = flexible
  - Multiple poses per ligand
- **Virtual screening**: evaluate **millions** of candidates computationally **LiGen** [1] – drug discovery platform:
  - Owned by Dompé Farmaceutici
  - Co-developed by PoliMi & CINECA
  - Used for Zika, SARS-CoV-2 drug discovery



LIGATE

[1] Andrea R. Beccari et al. "LiGen: A High Performance Workflow for Chemistry Driven de Novo Design". In: *Journal of Chemical Information and Modeling* 53.6 (2013), pp. 1518–1527

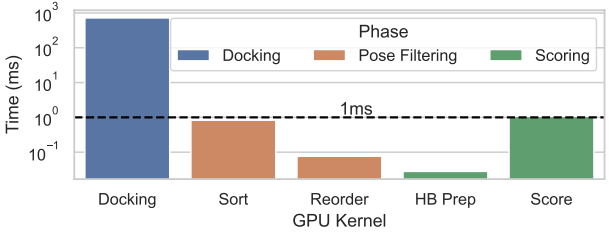
The LiGen Pipeline

- **CUDA**
  - LiGen-Latency
  - **LiGen-Batch**

LiGen-Batch [2], this work's focus:

- Each sub-group processes **one ligand**
- Ligand's atoms are assigned to the subgroup threads
- Multiple ligands per kernel → high throughput

Docking kernel dominates:



Docking: >**99%** of GPU time

⇒ our optimization focus

[2] Emanuele Vitali et al. "GPU-optimized approaches to molecular docking-based virtual screening in drug discovery: A comparative analysis". In: *Journal of Parallel and Distributed Computing* 186 (2024), p. 104819. ISSN: 0743-7315. DOI: https://doi.org/10.1016/j.jpdc.2023.104819

The Portability Challenge

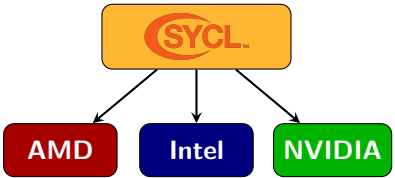
Problem:

- LiGen was **CUDA-only**
- Vendor-locked code cannot target the growing diversity of HPC accelerators

Solution:

- **SYCL** port [3]:
  - LiGen-Latency
  - LiGen-Batch (focus of this work)
- Single-source C++ for all GPU vendors

Portability is solved — but is a single implementation enough for good performance on different architectures?



[3] Luigi Crisci et al. "Enabling performance portability on the LiGen drug discovery pipeline". In: *Future Generation Computer Systems* 158 (2024), pp. 44–59

GPU Hardware Diversity

GPU architectures differ in fundamental ways:

NVIDIA GPUs

- SIMT execution (warps)
- Fixed warp size = 32 threads
- Fixed register allocation
- Tuned for one vendor

Intel Max GPUs

- Explicit SIMD vectorization
- **Configurable sub-group size** (16 or 32)
- **Configurable GRF** (128 or 256 regs)
- Tuning knobs that affect occupancy & spilling

A *portable* SYCL implementation alone is not enough — **architecture-specific tuning** is needed for peak performance.

LiGen SYCL Baseline – Limitations

SYCL solves vendor lock-in, but the baseline **ignores hardware diversity**:

1. No dynamic tuning

- Fixed work-group size (128)
- Fixed ligands per kernel (1120)
- Requires manual tuning per device

2. No Intel-specific opts

- Default sub-group size (32)
- Default GRF mode (auto)
- Misses Intel's configurable knobs

Known issue: LiGen-Batch suffers from **high register pressure**. The dock kernel allocates up to 60% more registers in SYCL than in CUDA, causing significant **register spilling** and limiting occupancy. [3]

Our goal: runtime heuristic for portable tuning + Intel-specific optimizations

[3] Luigi Crisci et al. "Enabling performance portability on the LiGen drug discovery pipeline". In: *Future Generation Computer Systems* 158 (2024), pp. 44–59

Proposed Approach

Contributions

1. **Performance-Portable SYCL Optimizations**

   Runtime heuristic using advanced SYCL/oneAPI features to dynamically adapt kernel workload to the target hardware – *no manual tuning required*

2. **Architecture-Specific Optimizations on Intel GPUs**

   Exploiting sub-group size tuning and GRF configuration to maximize performance on Intel Max GPUs

3. **Comprehensive Experimental Evaluation**

   Comparison of SYCL-baseline, SYCL-heur (portable), SYCL-intel (Intel-optimized), and SYCL-tuned (manual upper bound) with cross-platform roofline analysis against CUDA on NVIDIA V100S

LiGen SYCL Implementations

Version	Work-group	Sub-group	GRF	Lig./Kernel
SYCL-baseline	Fixed (128)	Default (32)	Compiler defined (auto)	Fixed (1120)
SYCL-heur	Heuristic (64–1024)	Default (32)	Compiler defined (auto)	Heuristic defined
SYCL-intel	Heuristic (64–1024)	Heuristic (16, 32)	Heuristic (small, large)	Heuristic defined
SYCL-tuned	Manually tuned (64–1024)	Manually tuned (16, 32)	Manually tuned (small, large)	Manually tuned (1000–4000)

- *SYCL-baseline*: lower bound (manual occupancy calculator)
- *SYCL-heur*: portable, automatic tuning – no hardware expertise needed
- *SYCL-intel*: extends heuristic with Intel-specific features
- *SYCL-tuned*: upper bound (exhaustive manual profiling)

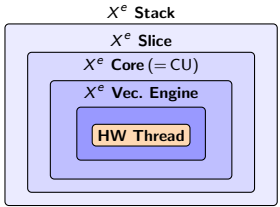
Performance-Portable Heuristic (SYCL-heur)

Goal: Automatically determine optimal # ligands per kernel at runtime.

Core formula [4]: $I = b \times CU \times \frac{wgs}{sgs}$

I = # ligands per kernel
b = max work-groups per CU
CU = compute units (CUDA SMs, Intel X^e Cores)
wgs = work-group size
sgs = sub-group size

Strategy:
Maximize ligand throughput → then minimize register spilling.



[4] Gianmarco Accordi et al. "Out of kernel tuning and optimizations for portable large-scale docking experiments on GPUs".
In: *The Journal of Supercomputing* 80.8 (2024), pp. 11798–11815

Runtime Heuristic Algorithm

Algorithm 1: *SYCL-heur*: Optimizing lig- and throughput

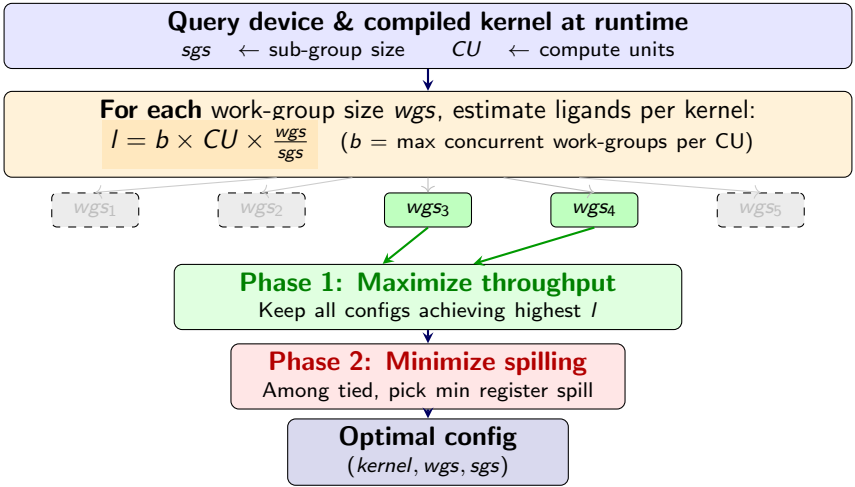
Input: device, slm_per_lig, kernel, wgs_vec
Output: opt_config

1 $sgs \leftarrow kernel.get\_sub\_group\_size()$;
2 $max\_lig\_kern \leftarrow 0$, $configs \leftarrow \emptyset$;
3 $cu \leftarrow device.num\_compute\_units()$;
4 **for** $wgs \in wgs\_vec$ **do**
5 $\quad slm\_wg \leftarrow slm\_per\_lig \cdot (wgs/sgs)$;
6 $\quad lig\_kern \leftarrow max\_wg\_per\_cu(\ldots) \cdot cu \cdot \frac{wgs}{sgs}$;
7 $\quad$ **if** $lig\_kern > max\_lig\_kern$ **then**
8 $\quad\quad max\_lig\_kern \leftarrow lig\_kern$;
9 $\quad\quad configs \leftarrow \{(kernel, wgs, sgs)\}$;
10 $\quad$ **else if** $lig\_kern = max\_lig\_kern$ **then**
11 $\quad\quad configs \leftarrow configs \cup \{(kernel, wgs, sgs)\}$;
12 $opt\_config \leftarrow \arg\min_{c \in configs} c.get\_spilled\_register()$;
13 **return** opt_config;

Key steps:

- **L1** Query default SG size from compiled kernel
- **L3** Query # CUs (portable across vendors)
- **L4–11** For each valid WGS, compute max ligands via the formula; keep best configs
- **L6** `max_wg_per_cu`: estimates concurrent WGs from SLM, registers, HW limits
- **L12** Tiebreaker: minimize **register spilling**

How the Heuristic Works

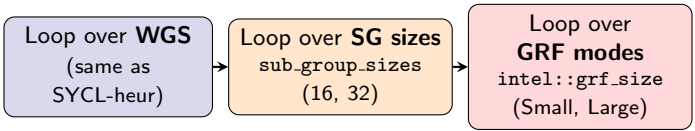


From SYCL-heur to SYCL-intel

SYCL-heur is portable but uses **fixed defaults** on Intel GPUs:

- Default sub-group size (SG=32 on Intel Max)
- Compiler-chosen GRF mode (auto) → not always optimal for register-heavy kernels

SYCL-intel extends the heuristic loop with **two additional dimensions**:

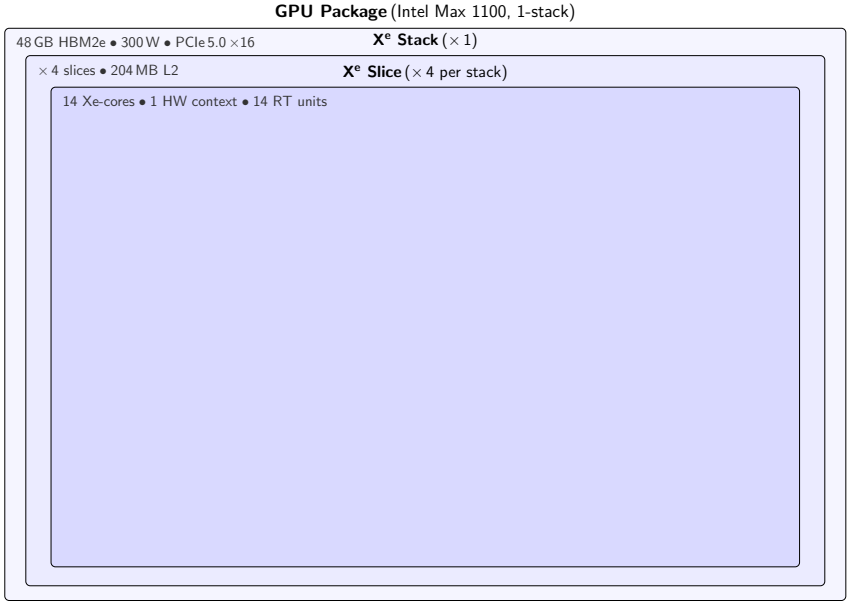


- Kernel **recompiled** for each SG size via `intel::reqd_sub_group_size`
- For each **(WGS, SG, GRF)** combination: evaluate throughput & spilling
- Systematically explores the full configuration space **without manual tuning**

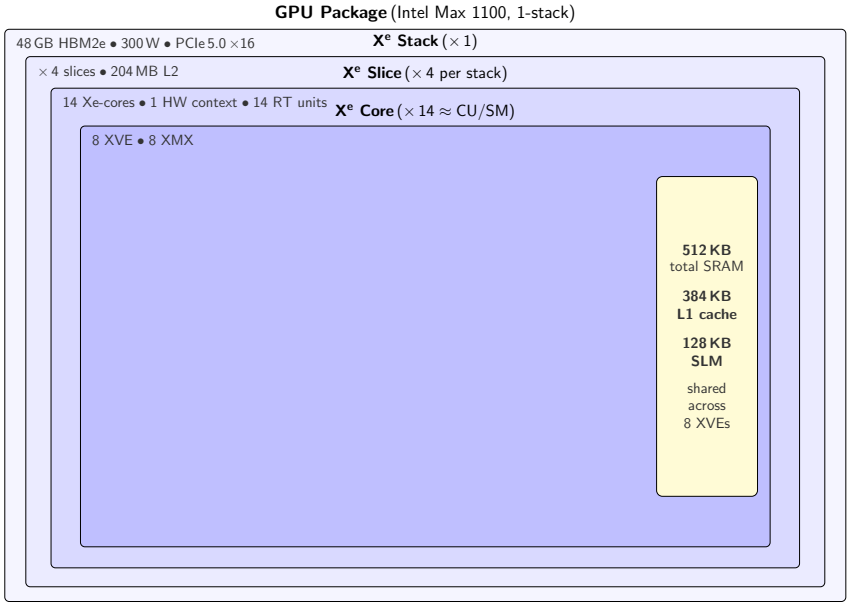
Sub-group Size $\times$ GRF Interplay

Sub-group Size $\times$ GRF Interplay

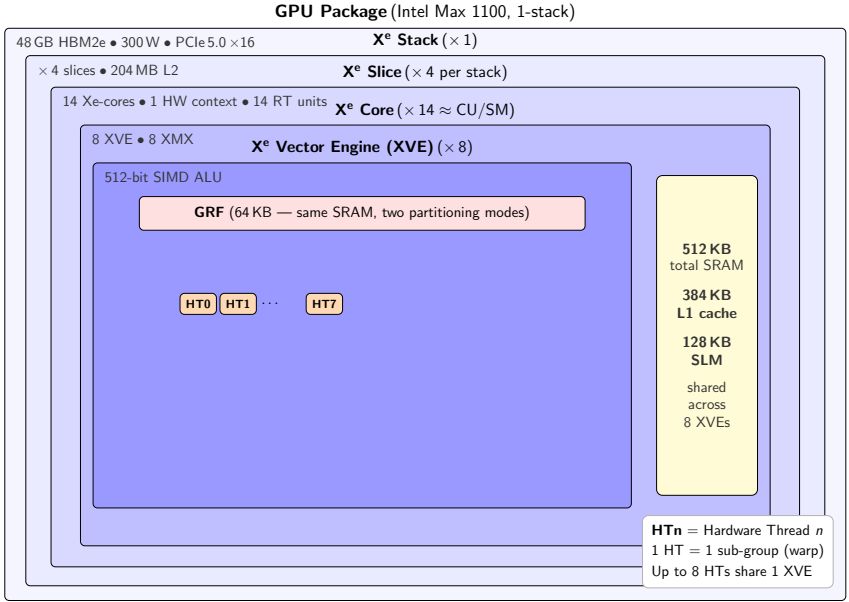
Sub-group Size × GRF Interplay



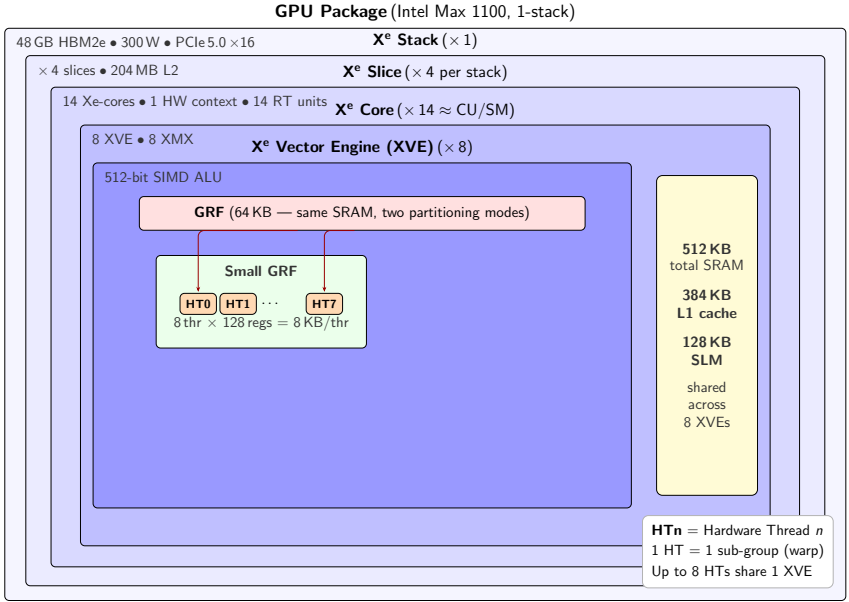
Sub-group Size × GRF Interplay



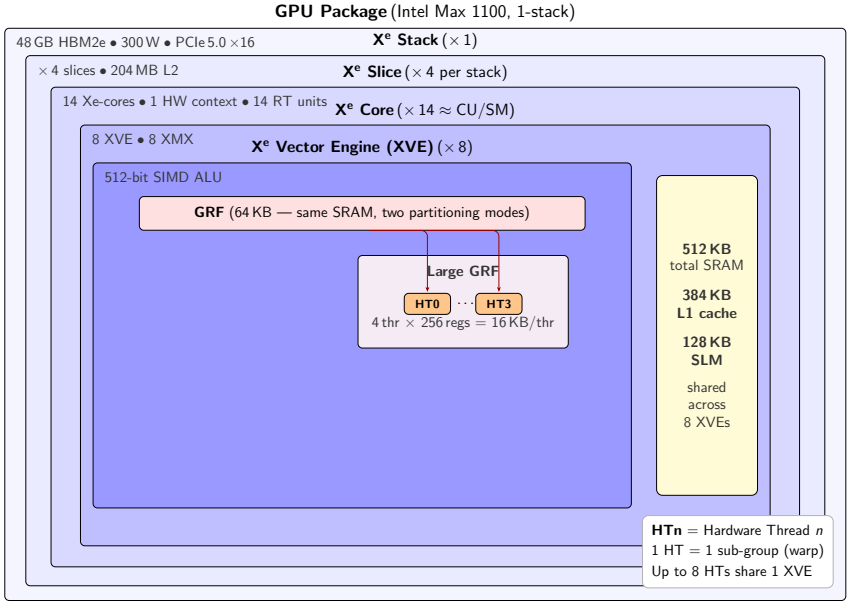
Sub-group Size × GRF Interplay



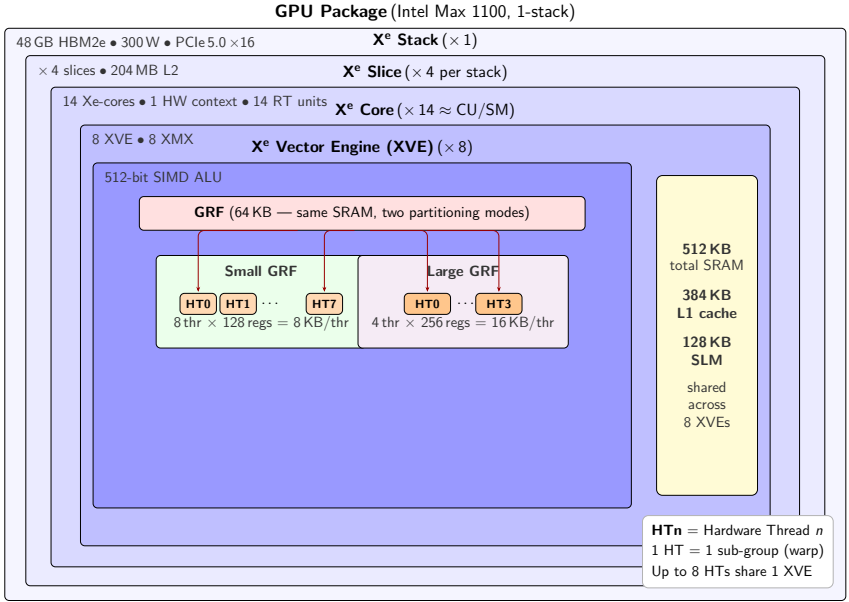
Sub-group Size × GRF Interplay



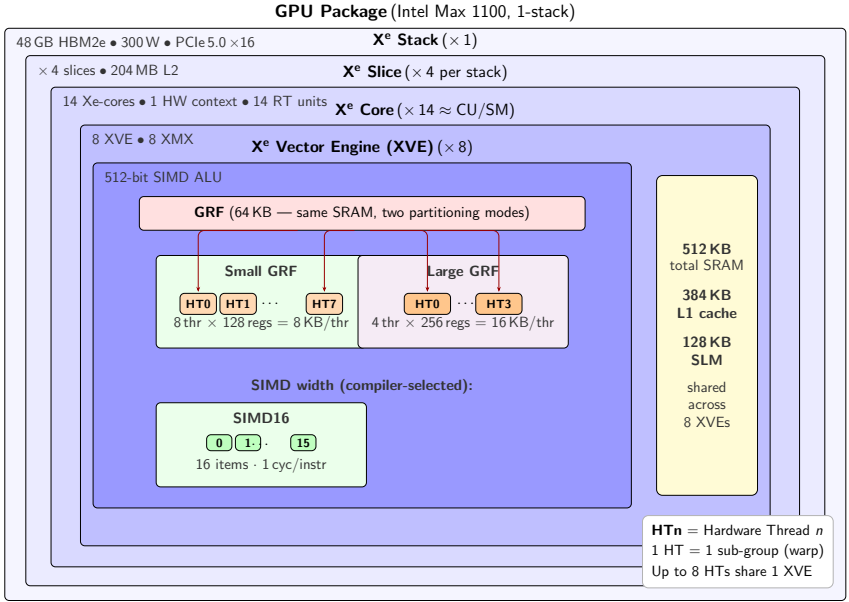
Sub-group Size × GRF Interplay



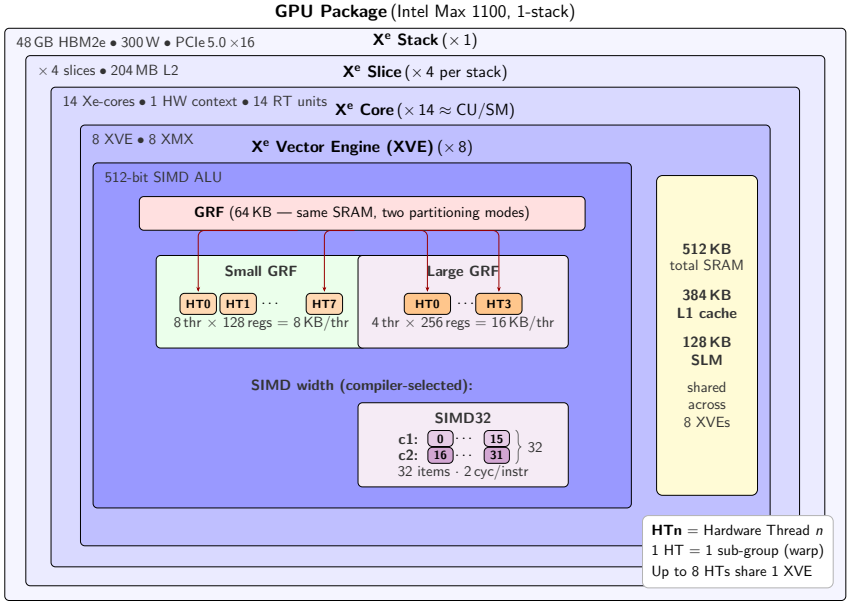
Sub-group Size × GRF Interplay



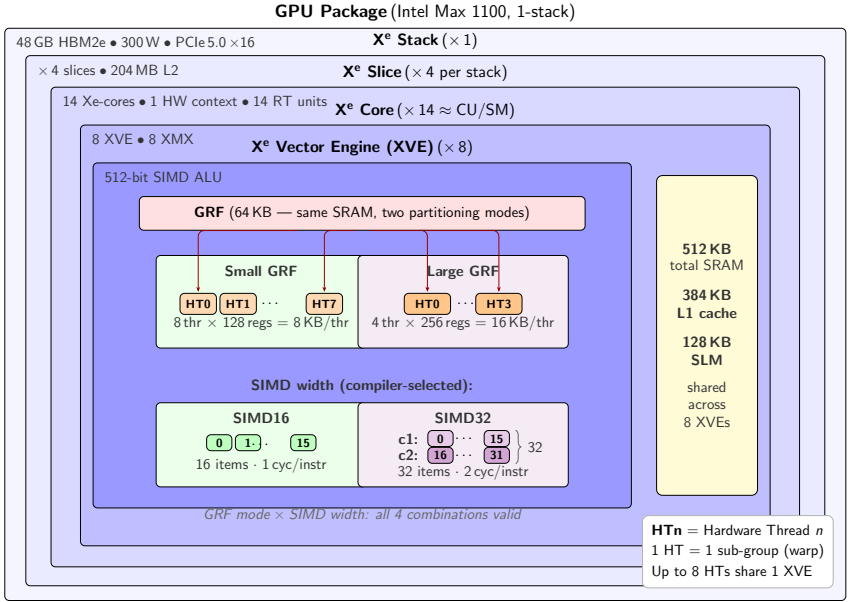
Sub-group Size × GRF Interplay



Sub-group Size × GRF Interplay



Sub-group Size × GRF Interplay



Sub-group Size × GRF Mode on X^e Vector Engine

1. GRF mode — partitions **1024 regs** per XVE among hardware threads:

- **GRF-Small**: 128 regs/HT ⇒ **8 HTs** / XVE *higher occupancy, more latency hiding*
- **GRF-Large**: 256 regs/HT ⇒ **4 HTs** / XVE *halved occupancy, less spilling*

2. SIMD width — sets sub-group size; does **not** change HT count *(each reg = 512 bit)*:

- **SIMD-16**: 16 work-items/HT, each FP32 var = $16 \times 32 = 512$ bits (**1 reg**) *1 cyc/instr*
- **SIMD-32**: 32 work-items/HT, each FP32 var = $32 \times 32 = 1024$ bits (**2 regs**) *2 cyc/instr*

  → **Halves effective variable capacity** with same register budget
  → Doubles work-items in flight

Strategy: Explore all SG × GRF combinations

Experimental Evaluation

Experimental Setup

Hardware

- **SYCL**: Intel Max 1100 GPU (CINECA)
  - Compiler: `icpx` (oneAPI 2025.1.0)
  - Profiler: Intel UniTrace
- **CUDA**: NVIDIA V100S GPU
  - Compiler: `nvcc` (CUDA 12.1)
  - Profiler: NVIDIA Nsight Compute

Datasets

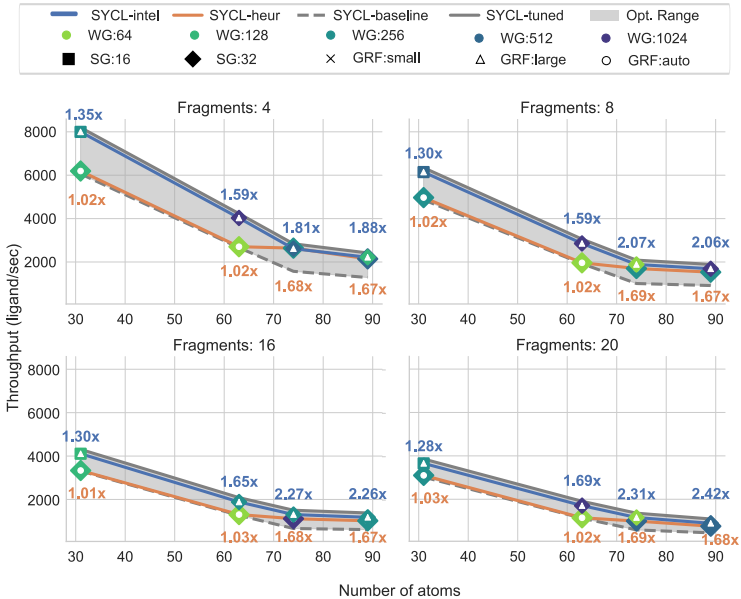
- 100,000 ligands per configuration
- Atom counts: 31, 63, 74, 89
- Fragment sizes: 4, 8, 16, 20
- 3 repetitions per experiment
- Median CoV[1]: 0.15% (stable)

16 configurations (4 atoms $\times$ 4 fragments)

Focus: Docking kernel (>99% of device time)

[1] **CoV:** Coefficient of Variation ($100 \times \frac{\sigma}{\mu}$)

Performance Results



- Ligands/s for different atom-fragment configurations
- Speedups are w.r.t. *SYCL-baseline*
- Gray region: improvement from Intel-specific optimizations

Key Performance Insights

SYCL-heur (portable):

- Geometric mean: **1.31**× vs baseline
- Selects better configs than manual occupancy calculator

SYCL-intel (Intel-optimized):

- Geometric mean: **1.76**× vs baseline
- Best case: **2.42**× (89 atoms, 20 fragments)
- Matches SYCL-tuned upper bound *without profiling*

Why it works:

- Trades occupancy for higher register availability
- Alleviates register spilling (primary bottleneck)
- Improves cache-hit ratio
- Best configs: SG=32 + GRF-Large

Example – 89 atoms, 20 fragments:

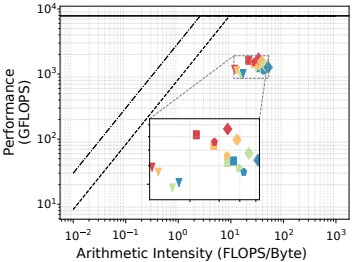
Baseline:	WG=256, 1120 lig/kern
SYCL-heur:	WG=512, 1792 → 1.68×
SYCL-intel:	+GRF-large → **2.42**×

Roofline Analysis

CUDA

NVIDIA V100S

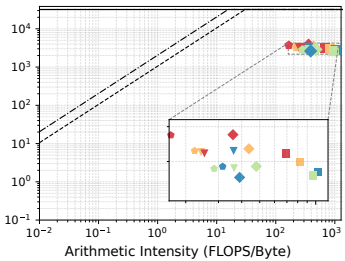
RE[1]: **16.4%**



SYCL-heur

Intel Max 1100

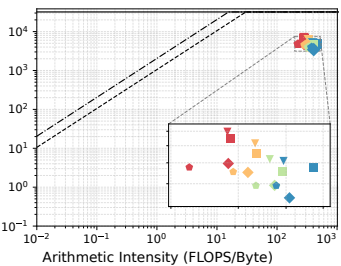
RE: **11.7%**



SYCL-intel

Intel Max 1100

RE: **21.8%**



GPU Memory Roof — L3 Roof — Compute Roof

Fragments: 4 — Fragments: 8 — Fragments: 16 — Fragments: 20

Atoms: 31 — Atoms: 63 — Atoms: 74 — Atoms: 89

[1] RE: Roofline Efficiency (geometric mean)

Conclusion

Conclusion

Summary:

- Portable heuristic (*SYCL-heur*) dynamically adapts kernel workload at runtime
  - Considers occupancy, register spilling, work-group size
  - Consistently outperforms baseline **without manual tuning**
- Intel-specific optimizations (*SYCL-intel*) deliver additional gains
  - Sub-group size tuning + GRF utilization $\rightarrow$ up to **2.42**$\times$
  - Closely matches manually-tuned upper bound
- Roofline efficiency: SYCL-intel (**21.8%**) $>$ CUDA (**16.4%**)

Key takeaway: Balance between **performance portability** and **architecture-specific optimizations** enables high performance without manual tuning.

Thank you! Questions?